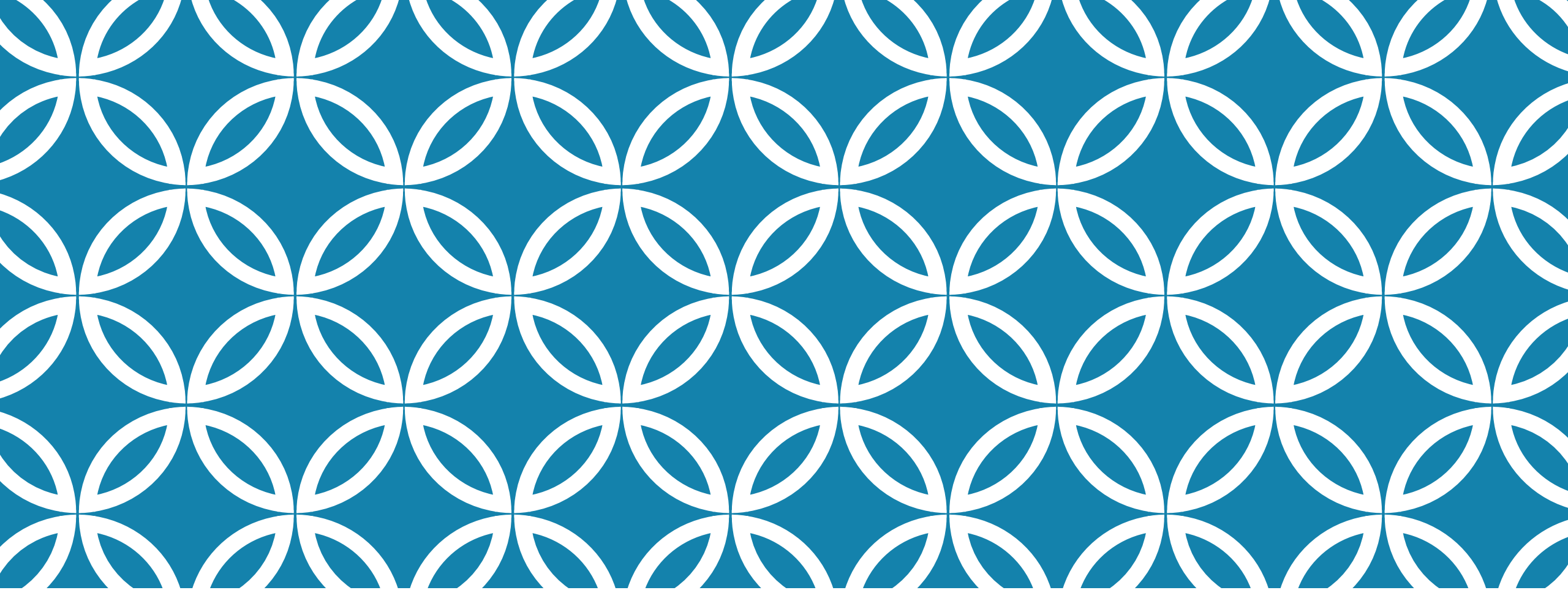


第二阶段 · 八个案例学习地图

前端实战 · 从一行静态文字到联网 App

学什么 · 为什么这样排 · 三条铁律 · 怎么学才不白学

八个案例，一屏看全

#	案例	一句话	新学的核心
1	静态个人名片页	把页面画出来	模板 / 样式 / 布局
2	动态待办清单	让页面动起来	ref / v-for / v-model
3	待办清单增强版	让数据自己算	computed / watch
4	多页面新闻列表	从一个页面到多个页面	路由 / 生命周期
5	新闻详情页	让手机听话	系统 API / 反馈
6	多功能计数器	把界面变成积木	组件化 / props
7	全局主题切换	把逻辑也复用起来	composable
8	新闻列表联网版	让数据从网上去取	uni.request

★ 八案例 = 八块地基，一块也不能缺

为什么是这个顺序：数据→渲染→数据移动

- 整条主线只有一句话：先让数据动起来，再让数据自己算，再让页面多起来，最后让数据走出去
- 案例 1 → 2：从「写死的页面」跨到「会变的数据」—— 这是 Vue 存在的全部理由
- 案例 3 → 5：数据开始「派生」和「联动」—— computed 算、watch 听、API 碰手机
- 案例 6 → 7：代码开始「被复用」—— 先复用界面（组件），再复用逻辑（composable）
- 案例 8：数据源从「本机」换成「互联网」—— 但页面结构一行都不用重写



★ 每一步都踩在前一步的地基上 —— 跳着学，后面必然塌

案例的痛点 → 解法

案例	痛点	案例的解法
1	页面是空白的，从哪下手?	模板骨架 + 样式 + Flex 布局
2	写死的内容改一条要改代码	ref 数据 + v-for 渲染
3	派生的东西手工算，还容易算错	computed 自动算 + watch 监听
4	一个页面塞不下所有内容	pages.json + 路由跳转 + 生命周期
5	App 只是「看」的，不会「做」	Toast / 拨号 / 剪贴板
6	同样的界面复制粘贴了三份	组件化 + props + emit
7	同样的逻辑也在复制粘贴	composable 组合式函数
8	数据写死的，改一条要重发版本	uni.request 去网上取数据

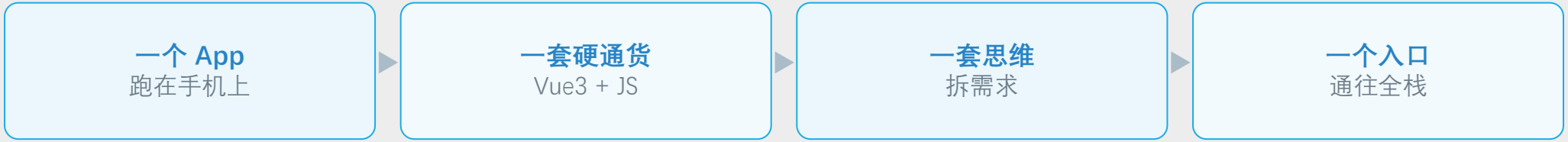
三条铁律

#	铁律	怎么做	为什么
1	只用 <script setup>	不写 export default	像 C 语言：引头文件 → 定义变量 → 导出接口
2	Flex 布局 + rpx 单位	杜绝 px 固定宽高	养成移动端适配的肌肉记忆
3	每案问一句「会崩吗」	动手做破坏性实验	错误演示比正确演示更记得住

- ★ 铁律 1：`<script setup>` 的直觉 —— 要用的东西必须先引入或定义，用完直接暴露给模板
- 对照传统写法：export default 里塞一堆 data / methods，还要想着 this 指向哪
- ★ 铁律 3 举例：案例 2 里 `v-for` 不写 `:key` 试试？案例 6 里 props 直接赋值试试？

完成八个案例的收获

- ① 一个能跑在手机上的完整 App —— 从名片到联网新闻，八次迭代的同一份工程
- ② 一套「换个项目还能用」的硬通货：ref / computed / watch / 组件 / composable
- 这些是 Vue3 和 JS 的东西，不是 UniApp 专有的 —— 换个框架、换个项目照样能用
- ③ 一套「拆需求」的思维：拿到任何界面，先问「数据是什么、它从哪来、它怎么变」
- ④ 一个通往全栈的入口 —— 数据可以从网上来，下一步就是自己写那个「网上」



前端：把页面点亮（案例 1–3）

案例	任务	最关键的一刻
1	一张个人名片页	第一次看到自己写的字出现在手机上
2	一个能增删的待办清单	★ 改数据，页面自己变 —— 不再操作 DOM
3	带筛选和统计的清单	★ 改数据，统计数字自己跟着变

- 这三案学的是「前端最小闭环」：数据 → 模板 → 界面
- 案例 2 的 `ref` 是整条主线的起点 —— 数据变了，界面跟着变
- 案例 3 的 `computed` 是第二次跃迁 —— 不用「动手算」，只要「描述关系」
- ⚠ 提醒：案例 2 是零基础学生第一次遇到的「思维断层」
- 以前写网页是「找到元素、改它」；现在是「改数据、让它自己变」—— 这一跃不跨过去，后面全乱

中段：让页面多起来、让手机动起来（案例 4–5）

案例	任务	最关键的一刻
4	列表页 + 详情页，可来回跳	★ 学会问「这段代码什么时候执行？」
5	能拨号、能复制的详情页	★ 手机真的动了 —— 拨号盘弹出来

- 案例 4 是全阶段最烧脑的一课：生命周期
- 同一个请求写在 `onLoad` 还是 `onShow`，表现完全不同 —— 一个只跑一次，一个每次回来都跑
- 这是零基础学生最容易埋雷的地方，也是后面每个案例都在偷偷用的知识
- 案例 5 是全阶段最有「爽感」的一课：调用手机能力
- 代码写下去，手机真的拨号了、剪贴板真的写入了
- ⚠ 案例 5 还藏了一个事故级细节：「点取消也拨号」

后段：把代码变成资产（案例 6–8）

案例	任务	最关键的一刻
6	一个能用三次的计数器组件	★ 不写 import，页面照样跑
7	一键切换的全局暗黑模式	★ 什么都没传，所有页面都变了
8	从网上取数据的三态新闻页	★ 断网了，界面也没崩

- 案例 6 的关键跃迁：从「写页面」到「造零件」
- 同一段代码用三次 —— 改一处，三处生效。这就是组件化的全部价值
- 案例 7 的关键跃迁：从「造零件」到「共享状态」
- 一个开关管全局，中间一根 props 都没传 —— 模块级 ref 的魔力
- 案例 8 的关键跃迁：数据源从本机换到互联网
- 页面结构一行没改，只是 `import` 换了个文件 —— 这才是「保留骨架、换掉业务」的威力

展望：网络请求

- **案例 8 被刻意做成「只做网络、不引入任何新 UI 和新路由」**——`async/await` +「后端返回的数据结构要一层层扒」是学习难点，严重影响对「前端渲染」的理解
- 页面就是案例 4 那个列表页，只把「数据从哪来」和「出错了怎么办」补上
- 掌握 `try/catch` + 三态 UI + 错误提示的「错误处理模板」

★ 下一阶段（UniCloud）：用云函数返回的数据，替换案例 4 的硬编码数据 —— 前端逻辑一行不用重写

怎么用这份地图

标记	含义
★	这一页是本案例最关键的转折点，上课时务必打起精神
⚠	这里有一个「听起来简单、实际会踩坑」的地方
`(0, '…')`	缩进项 —— 是对上一条的补充说明，口气更轻
配色	蓝 = 正面示范 / 绿 = 结论口诀 / 橙 = 警告与坑